UNIKLINIK KÖLN | BI-K Institute for Biomedical Informatics University of Cologne

Institute for Biomedical Informatics

WavePrep - Create Machine-Learning-Ready Datasets from MIMIC Waveforms

We propose WavePrep, an open-source software that provides a flexible pipeline for creating AI-ready datasets from the numeric and non-numeric MIMIC-III waveform database. In particular, we provide a user-friendly JSON interface that allows users to define preprocessing steps with fine-grained, step-by-step, and channel-specific control, perform a train-test split, and enable automatic parallelization to speed up dataset creation significantly.

https://github.com/BI-K/WavePrep

Mayra Elwes (mayra.elwes@uk-koeln.de), Oya Beyan , Ekaterina Kutafina

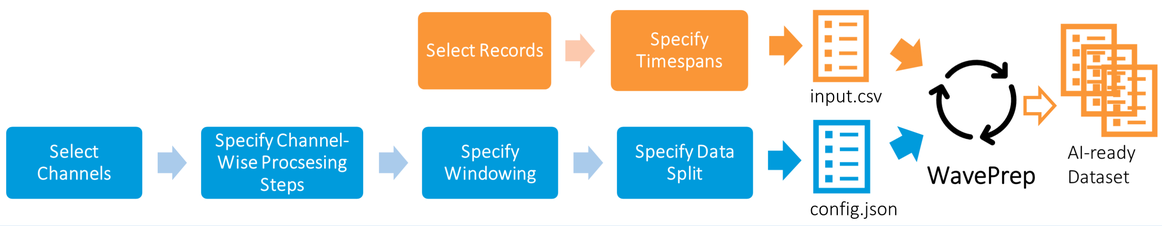
Introduction

The Medical Information Mart for Intensive Care (MIMIC) waveform databases provide high-resolution physiological signals from intensive care units but remain underutilized compared to the relational MIMIC core databases [1]. One barrier is the lack of accessible tools for transforming waveform data stored in the Waveform Database (WFDB) format into machine-learning- ready datasets. This work introduces WavePrep, an open-source software framework that addresses this gap by enabling flexible, transparent, and efficient preprocessing of MIMIC waveform data.

Methods

To use WavePrep, the user must provide two input files; an impression is given in Fig. 2. Firstly, a CSV file specifying the selected records, along with the time span for each record that should be included in the resulting dataset. Secondly, a JSON file specifying the channels to extract from the record, the channel-specific pre-processing steps, the windowing, and the split into train, test, and validation sets. WavePrep currently provides downsampling, data cleaning, imputation (details provided in Fig. 3) and windowing. WavePrep automatically determines when a preprocessing step, such as iterative imputation, requires defined train-test division, and will window and split the dataset before executing it. To avoid data leakage, a group-shuffle split at the patient-level is applied. The output of WavePrep is CSV files of the observation and prediction windows, is organised in a specific file structure, as shown in Fig. 4. WavePrep is developed in Python 3.12.3.

Figure 1: Overview on how to use WavePrep



The preprocessing step categories are implemented as abstract classes, enabling researchers to extend the framework. Records are processed in parallel.

Results

The processing speed of WavePrep is provided in Table 1. Compatibility with the non-numeric records of MIMIC III matched waveforms was confirmed via preprocessing that included downsampling and data cleaning of the plethysmography channel over the first 60 seconds of eight records. WavePrep was successfully used to prepare various datasets for a study benchmarking AI models for vital parameter prediction tasks [2], demonstrating its ability to create machine-learning-ready datasets.

Discussion

WavePrep generated the datasets for the MIMIC-III matched waveform v1.0 experiments as expected, demonstrating its ability to preprocess data to meet the needs of a machine learning project. By transforming data from WFDB to CSV, a format already familiar to many researchers, WavePrep may lower the barrier to working with the MIMIC waveform databases. The processing time per recorded hour decreases with increasing number of records to process, due to parallelisation in WavePrep. The open-source nature of the tool and the use of abstraction enable the community to adapt and further develop WavePrep to their specific preprocessing needs. The processing step categories are limited to downsampling, data cleaning, and imputing, for now, but can be expanded following the abstraction pattern to include further pre-processing categories. Using WavePrep addresses AI-readiness requirement through metadata by providing the config.json and input.csv files, which detail the processing and data split applied to the dataset.

Future Work

In future work, we will extend WavePrep with a detailed dataset quality report, further addressing AI-readiness and helping researchers create high-quality signal datasets with minimal coding. Additionally, we will integrate the Croissant metadata format for datatsets [3], addressing AI-readiness demands of the machine learning community.

Figure 2: Closer look at input.csv and config.json

Input.csv (red) specifies the selected records and timespans. Config.json (dark blue) specifies the desired processing steps.

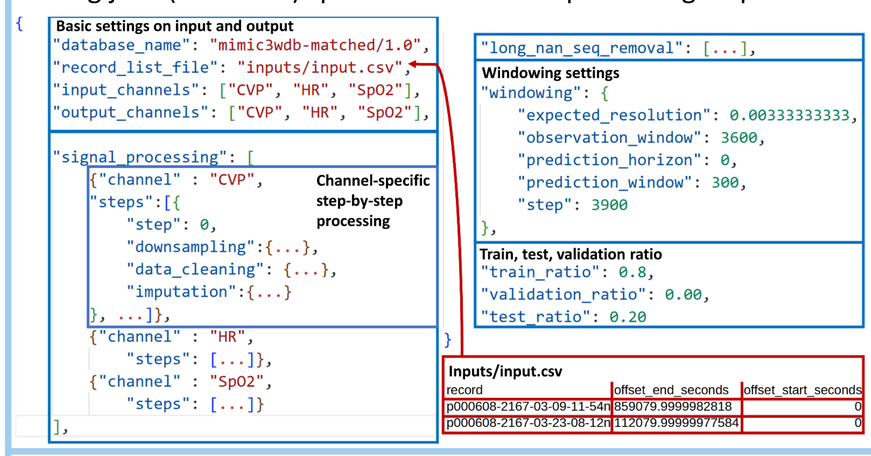


Figure 3: Signal processing specific configurations

Blue marks the supported processing types. The currently implemented strategies are listed in yellow. Orange marks the settings that the user must specify.

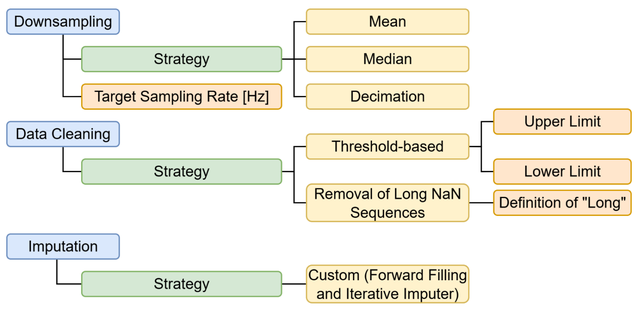


Figure 4: Output structure

At the first level, the resulting dataset is split into train, validation, and test sets. Within these folders, the processed samples are grouped by patient and named according to the rule record id sample number.csv. All observation windows of a patient's recordings are grouped under that patient's observation folder; likewise, the prediction windows are grouped under the prediction folder. Within the CSV files, one column corresponds to one channel, and each row corresponds to a timepoint.

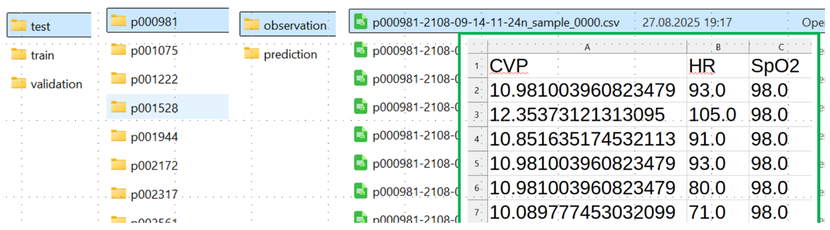


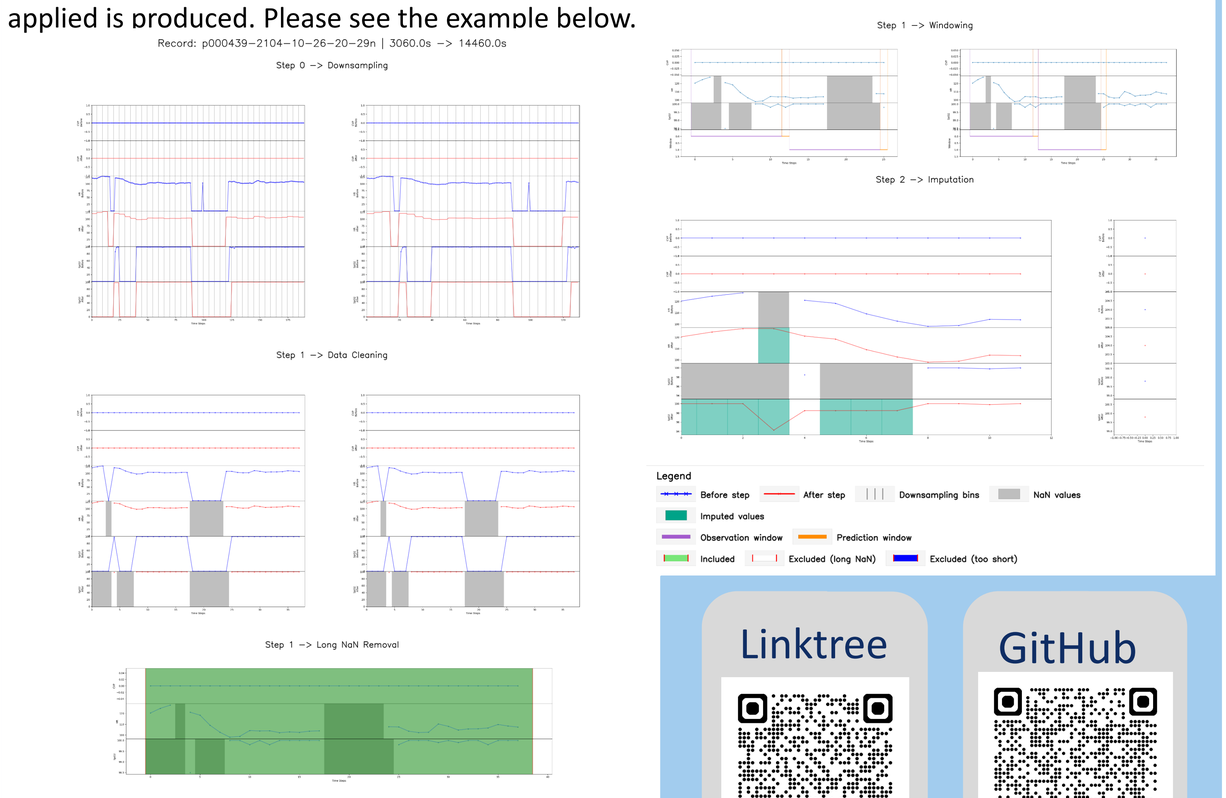
Table 1: WavePrep processing speed

We assess WavePrep's processing speed by creating eight datasets. All records are selected from the numeric records of MIMIC-III matched waveform v1.0. The selected records and time spans specified in input.csv differ across datasets, and the number of recordings and total duration to be processed for each dataset are shown below. The preprocessing steps specified in config.json are the same for all datasets. Six channels are extracted, and four processing steps are applied. The pipeline requires training of an iterative imputer. The recordings are sliced in a non-overlapping manner. The experiments are performed on a Server with two Intel Xeon Silver 4316 (40 cores, 80 threads @2.30GHz).

Data-set	# Records	Total duration of included timespans [s]	Processing time [s]	Processing time/# records [s]	Processing time/hour of recording [ms]
1	1092	78657.67	2603.95	2.38	33.10
2	945	54629.98	1853.87	1.96	33.94
3	456	42522.57	1595.46	3.50	37.52
4	512	36965.40	1430.72	2.79	38.70
5	565	35293.08	1414.06	2.50	40.07
6	445	21335.88	889.84	2.00	41.71
7	440	23041.47	968.62	2.20	42.04
8	241	14184.00	716.69	2.97	50.53

Figure 5: Visual processing report

20 user-selected records are randomly selected, and a visual report of each processing step applied is produced. Please see the example below.



Linktree

GitHub

References

[1] A. E. W. Johnson et al., MIMIC-III, a freely accessible critical care database. Sci Data, vol. 3, no. 1, p. 160035, May 2016. doi:10.1038/sdata.2016.35

[2] M. Elwes et al., Distribution Shift Analysis in Generalizable Modelling: Intensive Care Time-Series Data, Submitted for publication at MIE 2026. doi: 10.13140/RG.2.2.14589.83684

[3] M. Akhtar et al., Croissant: A Metadata Format for ML-Ready Datasets. In: Proceedings of the Eighth Workshop on Data Management for End-to-End Machine Learning. Santiago AA Chile: ACM; 2024. p. 1–6. doi: 10.1145/3650203.3663326

Workshop Biosignals 2026
Innsbruck, 25.02. – 27.02.2026